

Claude Code完全教程【视频文档】

一、初步上手

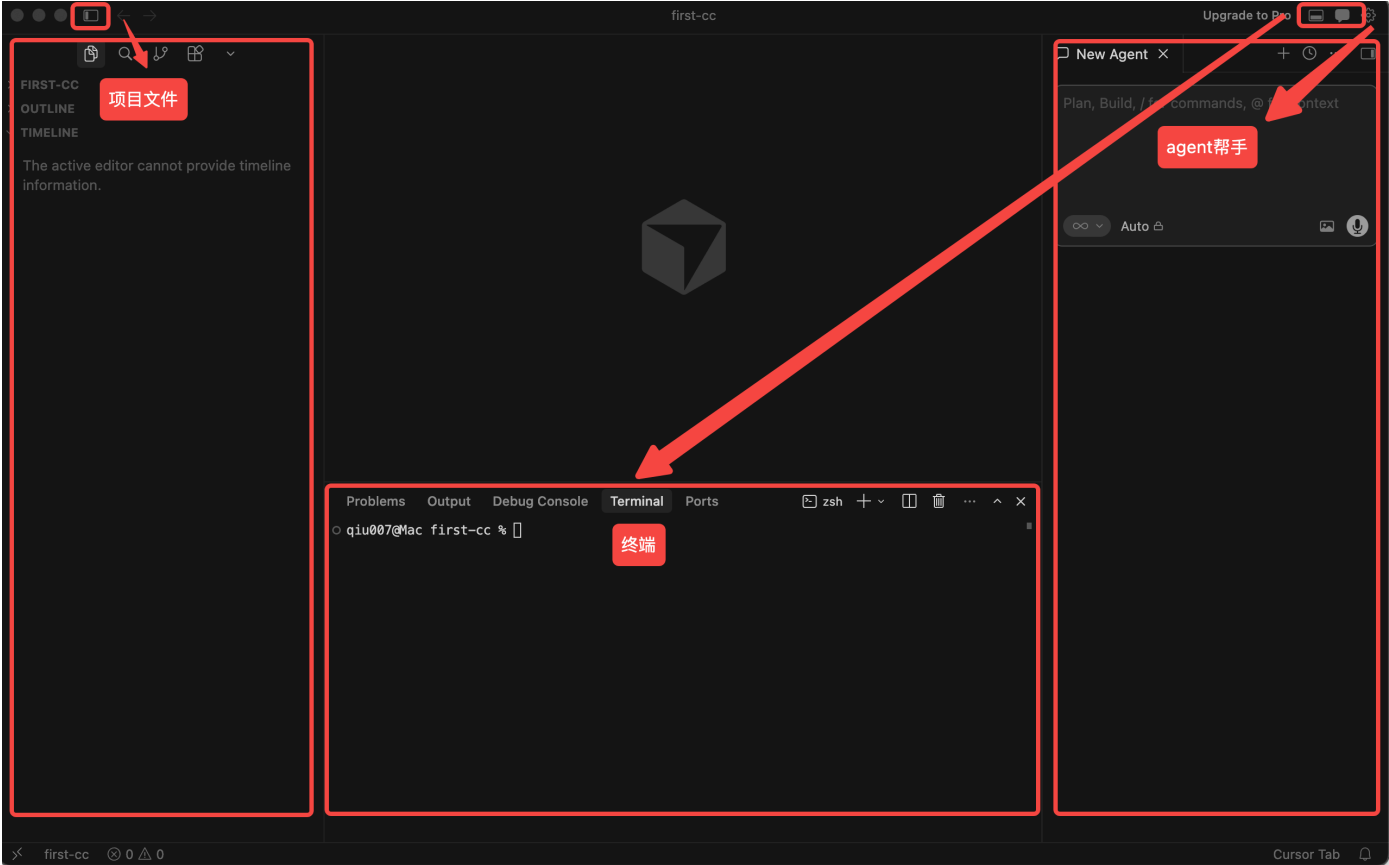
1. 部署前准备

首先需要下载和安装一个 **Agent IDE**（智能体集成开发环境）。

推荐使用以下免费IDE，无需购买会员，使用免费额度即可完成后续CC部署：

Agent IDE	下载地址
Cursor	官网下载
Google Anti-gravity	官网下载
Trae	官网下载（cn）
	官网下载（en）

以 **Cursor** 为例，下载进入后，可以构建如图的开发环境布局：



2. 安装Claude Code

方式 1：终端命令行安装（需魔法上网）

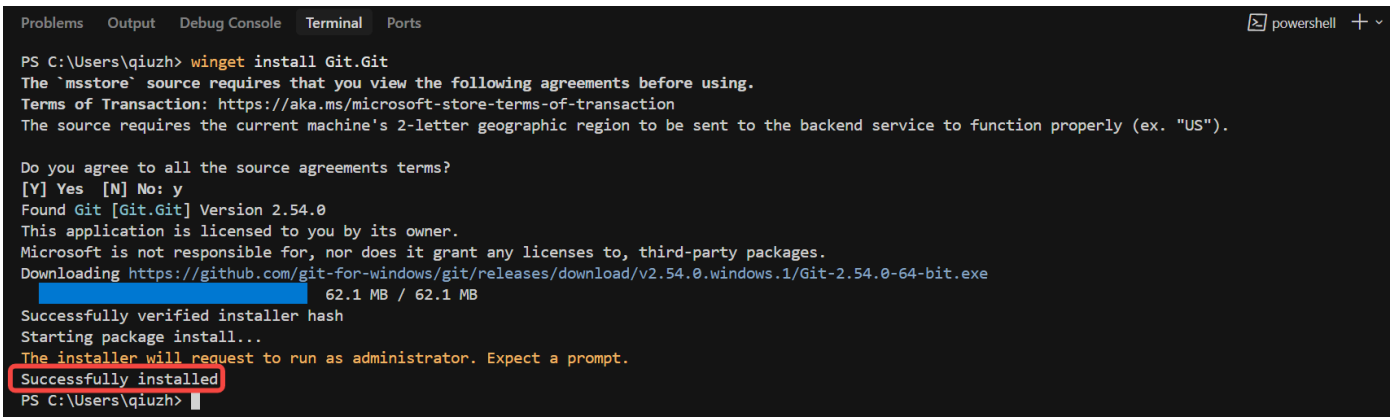
根据自身系统，选择对应代码，在IDE终端输入并回车：

Windows：

步骤 1：在ide终端用WinGet先安装Git，遇到选项直接输入 **y**（yes）

代码块

```
1 winget install Git.Git
```



```
Problems Output Debug Console Terminal Ports
PS C:\Users\qiuzh> winget install Git.Git
The 'msstore' source requires that you view the following agreements before using.
Terms of Transaction: https://aka.ms/microsoft-store-terms-of-transaction
The source requires the current machine's 2-letter geographic region to be sent to the backend service to function properly (ex. "US").

Do you agree to all the source agreements terms?
[Y] Yes [N] No: y
Found Git [Git.Git] Version 2.54.0
This application is licensed to you by its owner.
Microsoft is not responsible for, nor does it grant any licenses to, third-party packages.
Downloading https://github.com/git-for-windows/git/releases/download/v2.54.0.windows.1/Git-2.54.0-64-bit.exe
62.1 MB / 62.1 MB
Successfully verified installer hash
Starting package install...
The installer will request to run as administrator. Expect a prompt.
Successfully installed
PS C:\Users\qiuzh>
```

步骤 2：在ide终端输入代码，下载Claude code

- Windows - PowerShell（ide终端默认用这个）：

代码块

```
1 irm https://claude.ai/install.ps1 | iex
```

- Windows - CMD：

代码块

```
1 curl -fsSL https://claude.ai/install.cmd -o install.cmd && install.cmd && del install.cmd
```

- 下载好以后进行版本验证，版本号正常显示，代表安装完成！

版本号验证

```
1  claude --version
```

macOS:

- 直接在ide终端输入以下代码

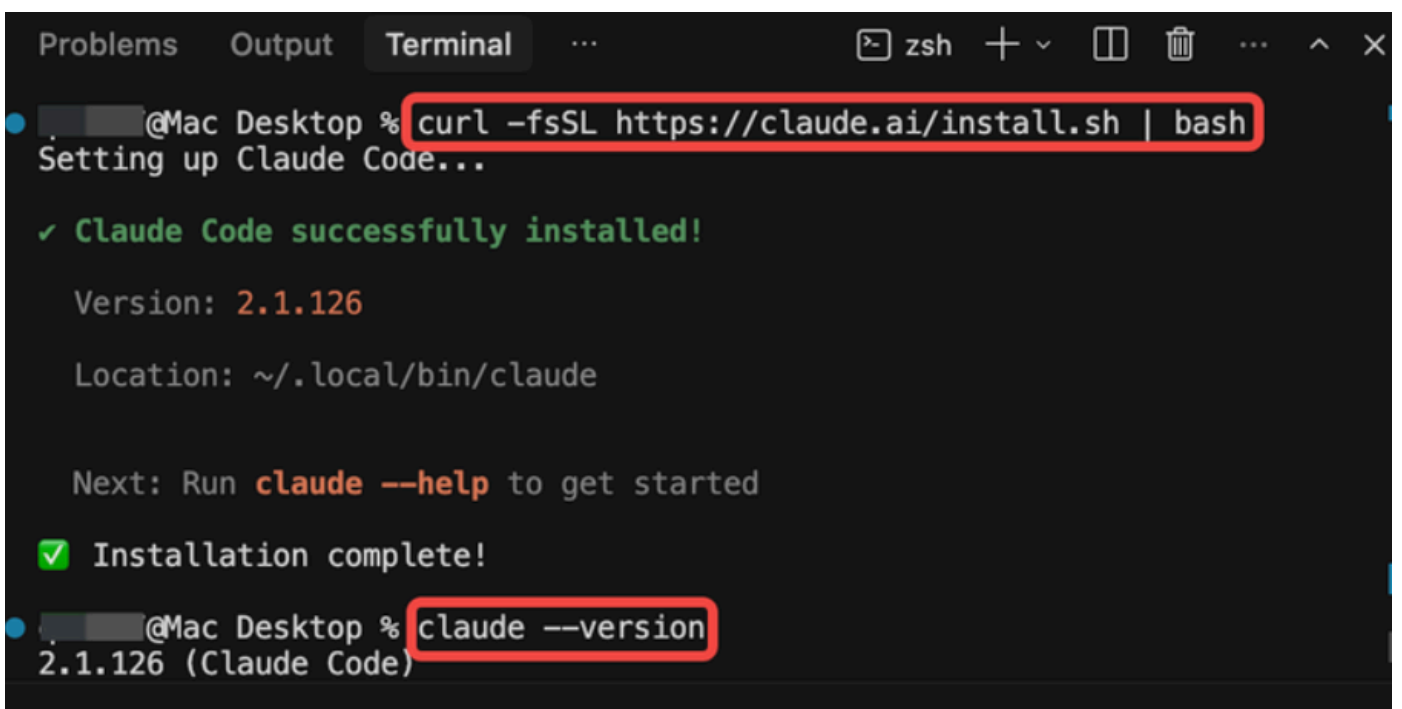
macOS / Linux

```
1  curl -fsSL https://claude.ai/install.sh | bash
```

- 下载好以后进行版本验证，版本号正常显示，代表安装完成!

版本号验证

```
1  claude --version
```



The screenshot shows a terminal window with the following content:

```
Problems  Output  Terminal  ...  zsh  +  -  [ ]  [ ]  ...  ^  x

@Mac Desktop % curl -fsSL https://claude.ai/install.sh | bash
Setting up Claude Code...

✓ Claude Code successfully installed!

Version: 2.1.126

Location: ~/.local/bin/claude

Next: Run claude --help to get started

✓ Installation complete!

@Mac Desktop % claude --version
2.1.126 (Claude Code)
```

方式 2: Agent原生安装（需魔法上网）

步骤 1: 直接向IDE Agent输入提示词:

代码块

```
1  帮我安装node并且用npm安装好最新的claude code
```

步骤 2: 验证Claude code版本号，确认安装成功

代码块

```
1  claude --version
```

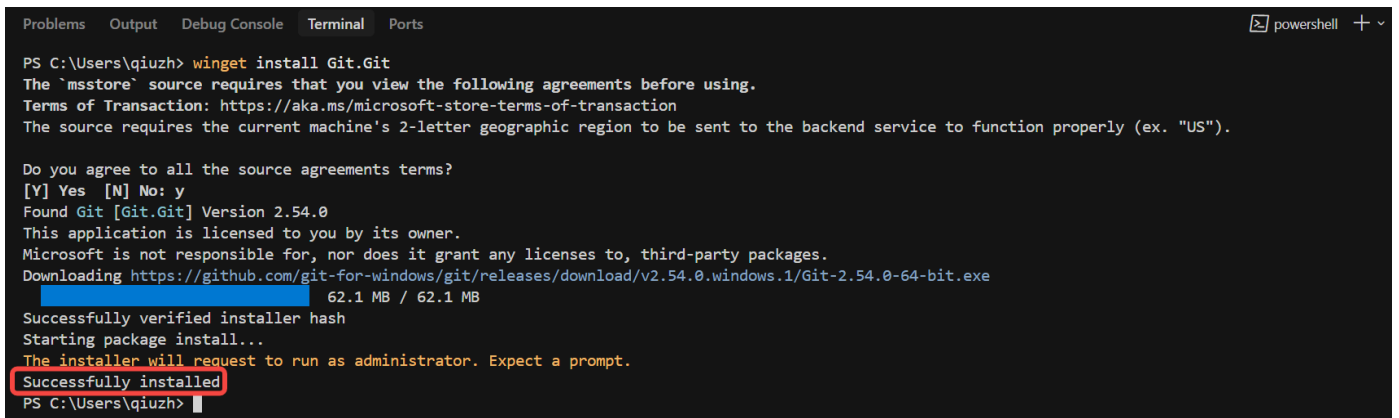
方式 3：无魔法安装Claude code

Windows:

步骤 1：用WinGet先安装Git，遇到选项直接y（yes）

代码块

```
1  winget install Git.Git
```



```
Problems Output Debug Console Terminal Ports powershell + v
PS C:\Users\qiuzh> winget install Git.Git
The 'msstore' source requires that you view the following agreements before using.
Terms of Transaction: https://aka.ms/microsoft-store-terms-of-transaction
The source requires the current machine's 2-letter geographic region to be sent to the backend service to function properly (ex. "US").

Do you agree to all the source agreements terms?
[Y] Yes [N] No: y
Found Git [Git.Git] Version 2.54.0
This application is licensed to you by its owner.
Microsoft is not responsible for, nor does it grant any licenses to, third-party packages.
Downloading https://github.com/git-for-windows/git/releases/download/v2.54.0.windows.1/Git-2.54.0-64-bit.exe
62.1 MB / 62.1 MB
Successfully verified installer hash
Starting package install...
The installer will request to run as administrator. Expect a prompt.
Successfully installed
PS C:\Users\qiuzh>
```

步骤 2：在ide agent输入以下提示词，让agent一条龙搞定Claude code安装

代码块

- 1 执行这条代码安装Claude code: `winget install Anthropic.ClaudeCode`
- 2 安装完后，把Claude Code的可执行文件路径配置到系统环境变量的Path里。

步骤 3：装完后，重启ide，验证Claude code版本号，确认安装成功

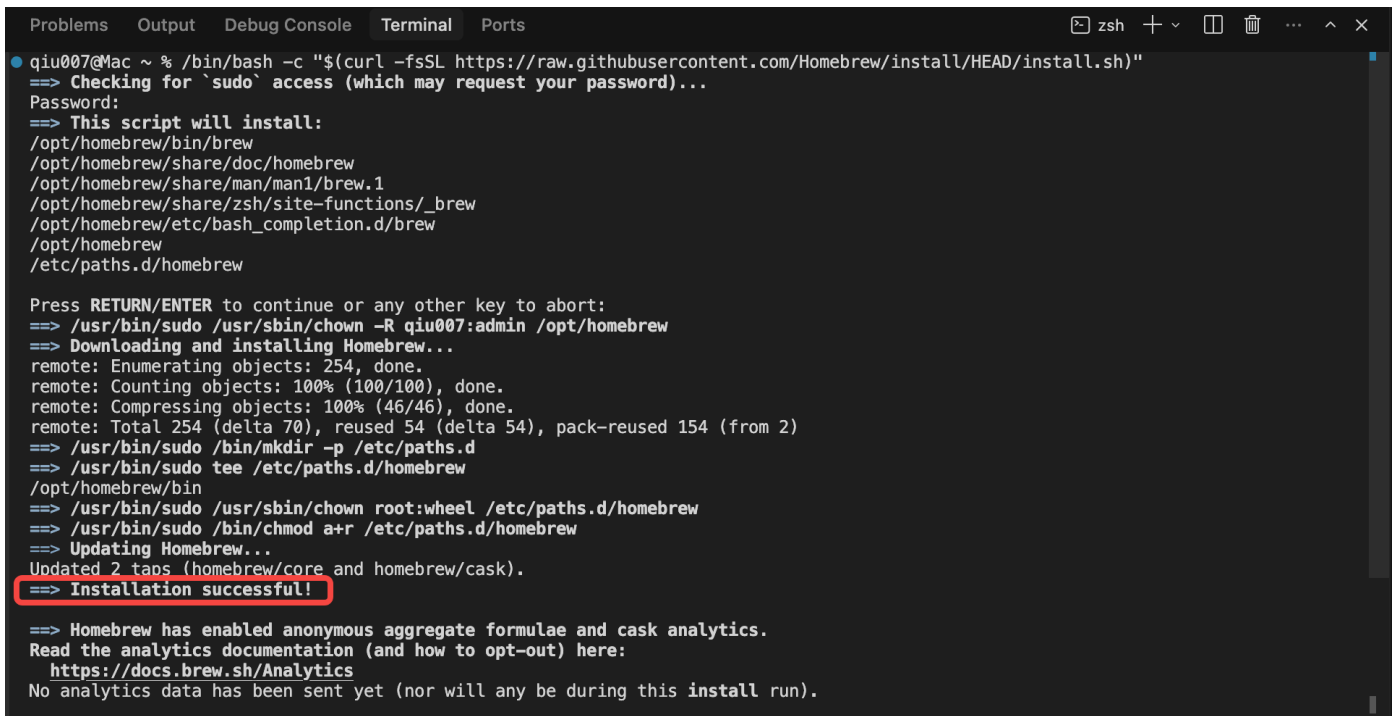
代码块

```
1  claude --version
```

macOS:

步骤 1：在ide终端先装Homebrew

```
1 /bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

A terminal window showing the installation of Homebrew on a Mac. The user runs a command to fetch the installation script from GitHub. The script checks for sudo access, lists the files it will install, and then proceeds to download and install Homebrew. It shows progress for enumerating, counting, and compressing objects. After creating the necessary directories and setting permissions, it updates the Homebrew taps. The installation is successful, and the user is informed that Homebrew has enabled anonymous aggregate formulae and cask analytics. The line "Installation successful!" is highlighted with a red box.

```
Problems Output Debug Console Terminal Ports
qiu007@Mac ~ % /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
==> Checking for `sudo` access (which may request your password)...
Password:
==> This script will install:
/opt/homebrew/bin/brew
/opt/homebrew/share/doc/homebrew
/opt/homebrew/share/man/man1/brew.1
/opt/homebrew/share/zsh/site-functions/_brew
/opt/homebrew/etc/bash_completion.d/brew
/opt/homebrew
/etc/paths.d/homebrew

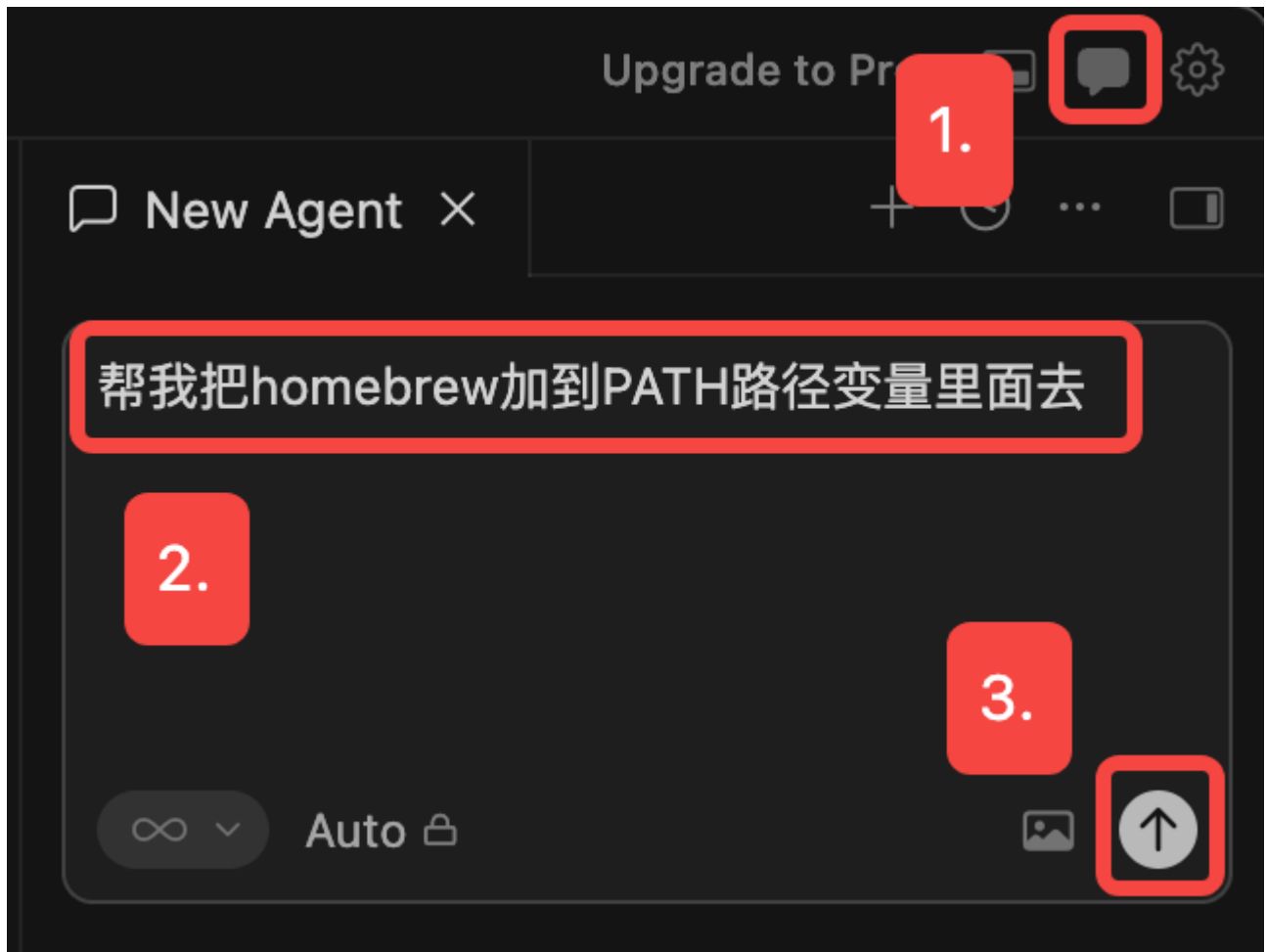
Press RETURN/ENTER to continue or any other key to abort:
==> /usr/bin/sudo /usr/sbin/chown -R qiu007:admin /opt/homebrew
==> Downloading and installing Homebrew...
remote: Enumerating objects: 254, done.
remote: Counting objects: 100% (100/100), done.
remote: Compressing objects: 100% (46/46), done.
remote: Total 254 (delta 70), reused 54 (delta 54), pack-reused 154 (from 2)
==> /usr/bin/sudo /bin/mkdir -p /etc/paths.d
==> /usr/bin/sudo tee /etc/paths.d/homebrew
/opt/homebrew/bin
==> /usr/bin/sudo /usr/sbin/chown root:wheel /etc/paths.d/homebrew
==> /usr/bin/sudo /bin/chmod a+r /etc/paths.d/homebrew
==> Updating Homebrew...
Updated 2 taps (homebrew/core and homebrew/cask).
==> Installation successful!

==> Homebrew has enabled anonymous aggregate formulae and cask analytics.
Read the analytics documentation (and how to opt-out) here:
https://docs.brew.sh/Analytics
No analytics data has been sent yet (nor will any be during this install run).
```

步骤 2: 向ide agent输入提示词:

代码块

```
1 帮我把homebrew加到PATH路径变量里面去
```



步骤 3: 在ide终端输入代码，安装Claude Code

代码块

```
1 brew install --cask claude-code@latest
```

步骤 4: 验证Claude code版本号，确认安装成功

代码块

```
1 claude --version
```

3. 配置大模型

1. 下载CC Switch做多模型切换和管理

😊 CC Switch的GitHub仓库下载页: <https://github.com/farion1231/cc-switch/releases/tag/v3.14.1>

最新版本号：v3.14.1

步骤1：在下载页，根据系统选择对应下载包：

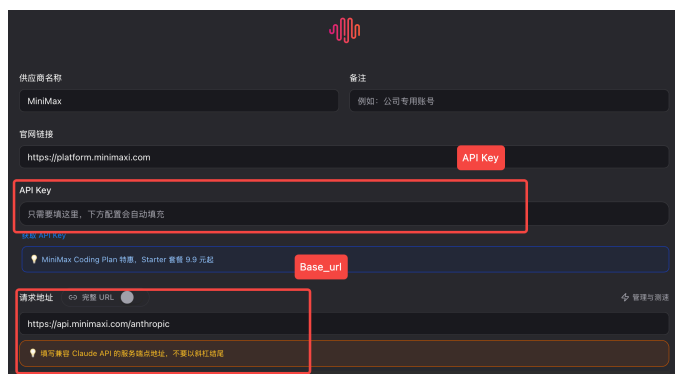
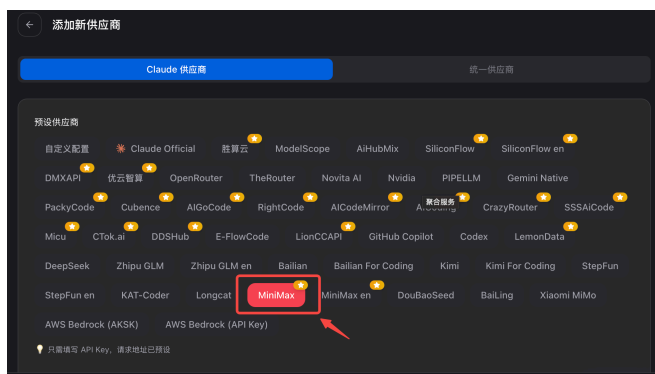
▼ 资源 18

CC-Switch-v3.14.1-Linux-arm64.ApplImage	sha256:b9188866dca0ec8c7a369c40d3...	84.3 MB	上周
CC-Switch-v3.14.1-Linux-arm64.ApplImage.sig	sha256:87836bac39dc43312d96dacfe6...	424 Bytes	上周
CC-Switch-v3.14.1-Linux-arm64.deb	sha256:984912f262a781b81ac2cad6c6...	10.9 MB	上周
CC-Switch-v3.14.1-Linux-arm64.rpm	sha256:19f66f480ff53d064013218156...	10.9 MB	上周
CC-Switch-v3.14.1-Linux-x86_64.ApplImage	sha256:a2e5c4183156437c96a1fe72df...	86.5 MB	上周
CC-Switch-v3.14.1-Linux-x86_64.ApplImage.sig	sha256:54097bd67697a687213aa4a2e6...	424 Bytes	上周
CC-Switch-v3.14.1-Linux-x86_64.deb	sha256:977d780d0400680d854ade98d9...	11.3 MB	上周
CC-Switch-v3.14.1-Linux-x86_64.rpm	sha256:d9b3370a467f5c54f51fe43ff7...	11.3 MB	上周
CC-Switch-v3.14.1-macOS.dmg	sha256:97f48ae9e04ff24defa34cd787...	23.4 MB	上周
CC-Switch-v3.14.1-macOS.tar.gz	sha256:c27774f9d67546807752b753df...	24 MB	上周
CC-Switch-v3.14.1-macOS.tar.gz.sig	sha256:3ace77c59e42dc208642ad6238...	408 Bytes	上周
CC-Switch-v3.14.1-macOS.zip	sha256:595cddb510405b12578ccc6250...	23.4 MB	上周
CC-Switch-v3.14.1-Windows-Powershell.ps1	sha256:3747d1218e1fc7f3671b61d1eb...	10.9 MB	上周
CC-Switch-v3.14.1-Windows.msi	sha256:82131ae0fb57d5ba5ab8d1b67c...	11.2 MB	上周
CC-Switch-v3.14.1-Windows.msi.sig	sha256:36fdc59090cbb5d0755c4c4030...	420 Bytes	上周
latest.json	sha256:7060129f4e5e271786c6e20da3...	2.89 KB	上周

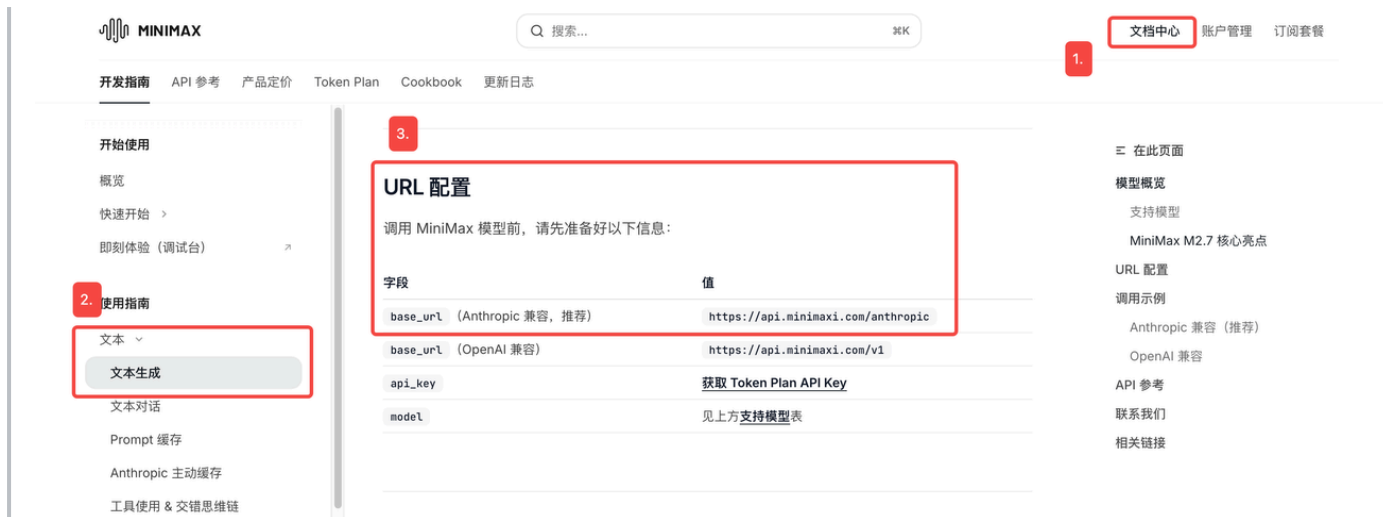
步骤2：安装完成后，务必在打开Claude Code之前，优先设置CC Switch。在CC Switch的Claude Code页面添加API Key供应商：



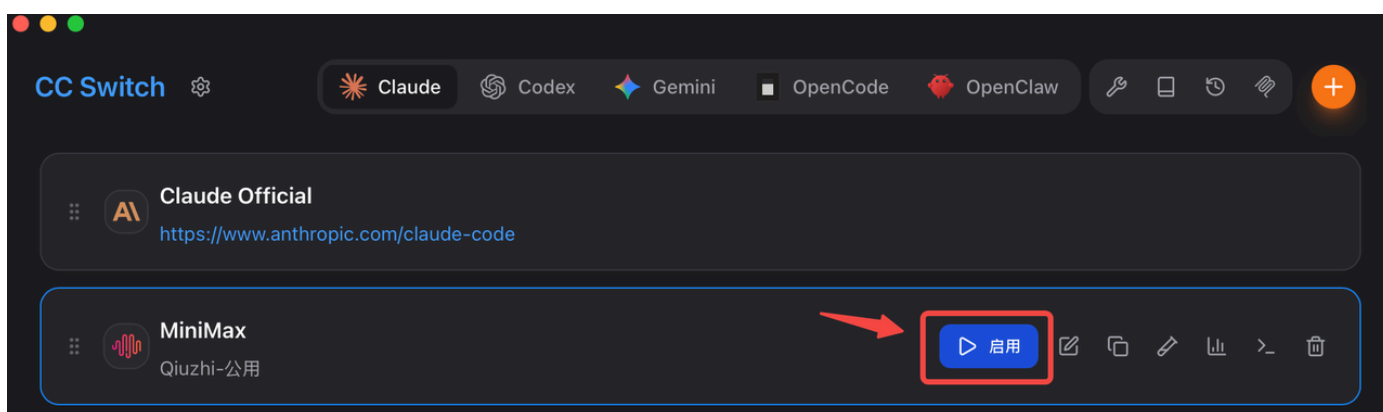
步骤3：以 Minimax 为例，选择对应厂商（中国版），然后填写API Key和Base URL：



- 如果不知道Base URL，可以在API Key的官方文档里找到（以Minimax的官方文档为例）



步骤4: 选择「启用」设置好的API, CC的大模型配置完成! 下一步可以准备打开Claude Code了:

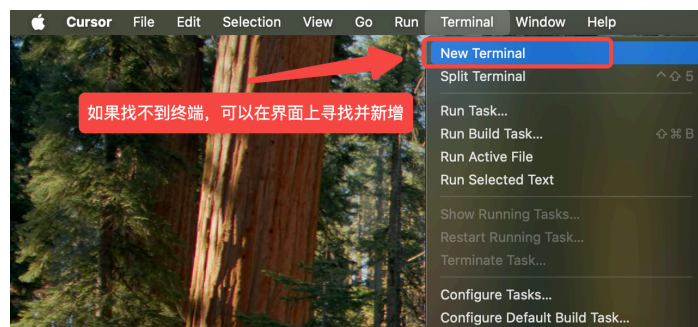
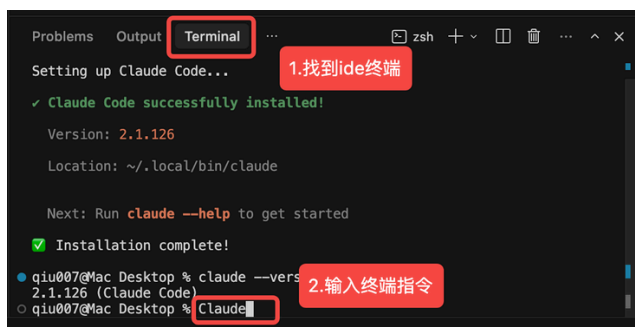


4. 基础实操

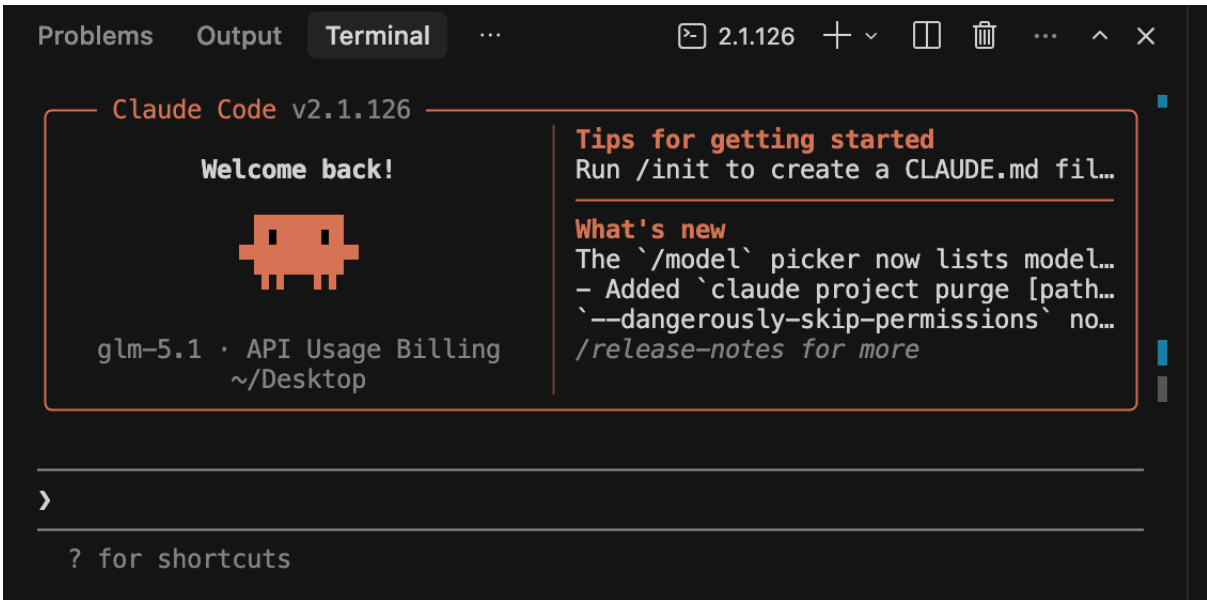
步骤1: 在IDE终端打开CC, 输入:

代码块

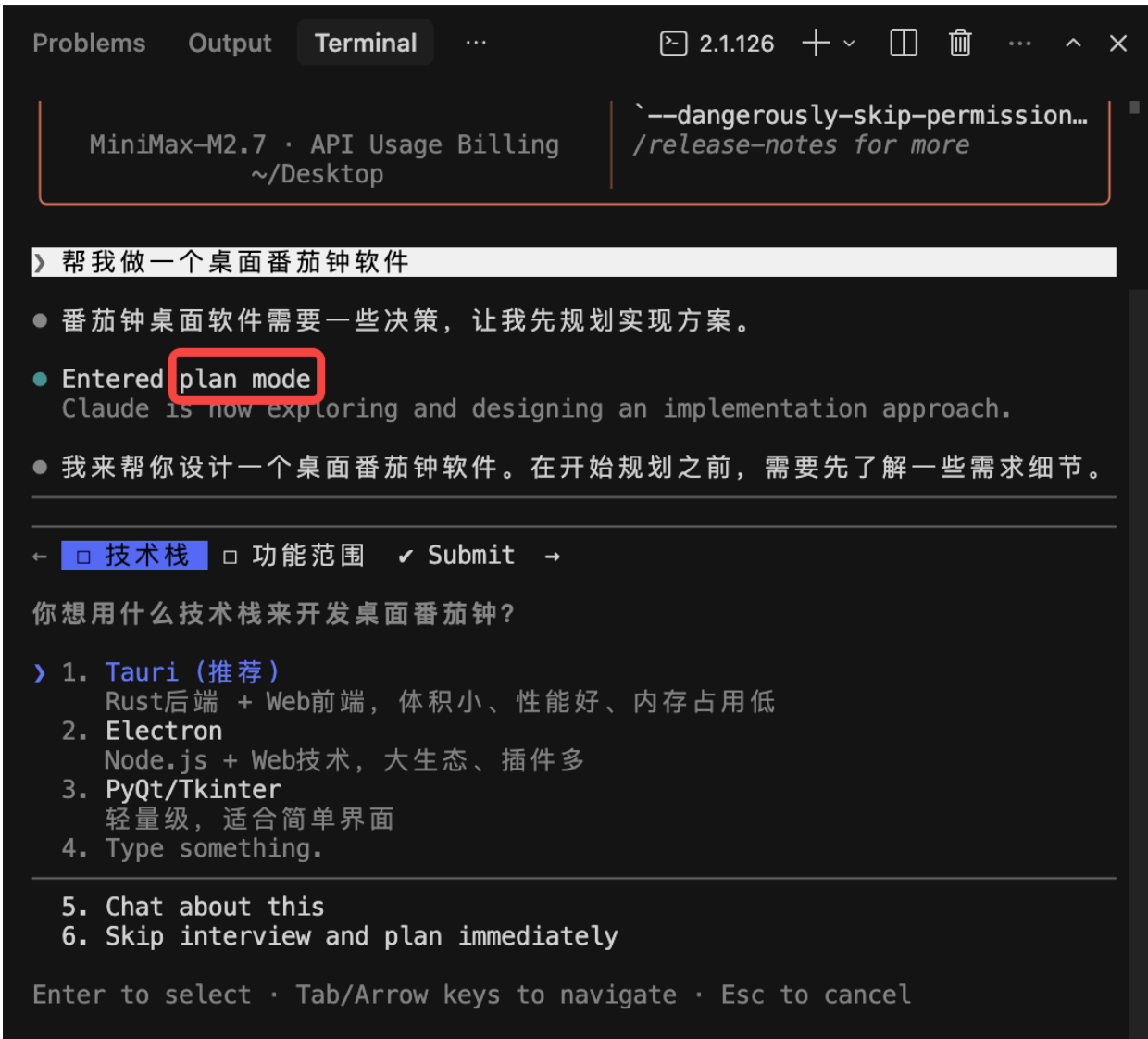
```
1 claude
```



步骤2: 根据个人喜好设置皮肤与主题, 然后一路 `yes` 后, 进入CC主界面:



步骤3：默认情况下，CC接到任务后会进入 计划模式（Plan Mode）：



步骤4：使用shift+tab，可以让CC在计划模式、默认模式和Accept Edits模式之间来回切换

模式	功能

默认模式	启动 Claude Code 后的初始交互状态，类似于一个增强版的对话终端，用于意图确认、信息查询和简单任务。
计划模式	Claude 进行复杂的代码更改时，它会进入这一阶段。分析需求并制定执行步骤。不直接动代码。
Accept Edits模式	这是 Claude Code 的“落地”阶段，会执行代码变更的最终确认与写入

额外提醒：

1. 如果希望CC能一路绿灯执行所有操作，需要输入指令 `/exit` 退出重启后，在启动CC时输入以下命令

在ide终端输入：

```
1  claude --dangerously-skip-permissions
```

2. CC在Accept Edits模式下，执行终端命令时，下图三个选项的含义依次分别是：

- 仅同意这一次的命令执行
- 同意，且该项目之后执行项目依赖安装时，不再询问
- 不同意，再商量

```
Bash command

cd /Users/qiu007/Desktop/pomodoro-pixel && pnpm install 2>&1 | tail
-30
安装依赖

Do you want to proceed?
> 1. Yes
2. Yes, and don't ask again for: pnpm install *
3. No
```

5. 如何提供文件给CC

方式 1：本地文件

使用 `@` 指令让CC进行本地文件信息查找：

```
> @/Users/qiu007/

/Users/qiu007/Applications/
/Users/qiu007/Desktop/
/Users/qiu007/Documents/
/Users/qiu007/Downloads/
/Users/qiu007/Library/
/Users/qiu007/Movies/
```

方式 2：图片

直接拖拽图片至对话框，或复制/粘贴：

- Windows: Alt + V
- macOS: Command + V

方式 3：多行文本输入

在CC文本框内换行的快捷键（不是 Shift + Enter）：

- Windows: Ctrl + Enter
- macOS: Option + Enter

6. 指令大全

Claude指令	功能说明
/help	提供所有指令，以及指令背后遵循的意思
/model	切换高中低档模型
/btw	By the way缩写，可以暂时切出正在执行的项目，隔离上下文，方便使用者与CC进行临时对话。会话完毕后，可按esc消除临时会话
/simplify	输入后会派生出3个agent，从代码质量、运行效率和复用性三个角度做一次代码审核，然后自动优化修改
/rewind	进入回滚界面
/compact	主动压缩精简上下文
/clear	彻底清空上下文，相当于重开一个会话

<code>/context</code>	详细展示agent当前的上下文信息，诸如：上下文占比，上下文类别等等
<code>/resume</code>	在全新的上下文窗口，选择恢复到之前的对话
<code>/init</code>	初始化创建项目级Claude.md
<code>/memory</code>	针对Claude的全局、项目记忆，以及auto memory进行操作和管理
<code>/agents</code>	创建、调用、管理子agent
<code>/plugin</code>	发现新插件，管理已下载插件，新增插件生态

更多指令可参考以下pdf文档：



claude code最新指令大全.pdf
133.36KB



二、掌控与管理

1. Git下载与设置

在CC中输入以下提示词，根据CC引导进行操作：

代码块

1 帮我下载Git，并与我的GitHub账号绑定

2.上下文管理

步骤1:确认上下文进度

- 输入 `/context` 命令，它会详细的展示我们的上下文占比信息：

代码块

1 /context

```
> /context
└─ Context Usage
   MiniMax-M2.7
   38.2k/200k tokens (19%)
   Estimated usage by category
   System prompt: 5.8k tokens (2.9%)
   System tools: 14.2k tokens (7.1%)
   MCP tools: 11.7k tokens (5.9%)
   Custom agents: 277 tokens (0.1%)
   Memory files: 53 tokens (0.0%)
   Skills: 2.6k tokens (1.3%)
   Messages: 4k tokens (2.0%)
   Free space: 128.4k (64.2%)
   Autocompact buffer: 33k tokens (16.5%)
```

步骤2:主动压缩上下文

- cc会在上下文即将满的时候，会帮我们自动压缩
- 我的习惯是看到高于60%了，就大概率需要 `/compact`

代码块

```
1 /compact
```

步骤3:彻底清空上下文

代码块

```
1 /clear
```

步骤4:context占用条常驻

- 对CC输入提示词，并根据CC引导完成后重启终端即可：

提示词

```
1 帮我配一个 statusLine,显示当前目录+模型+上下文剩余百分比
```

```
> 帮我配一个 statusLine,显示当前目录+模型+上下文剩余百分比
• statusline-setup(Setup statusLine config)
  └─ Done (4 tool uses · 4.8k tokens · 40s)
    (ctrl+o to expand)

• 配置完成！状态行现在显示：

qiu007 | Claude 3.5 Sonnet | 87%

格式： 目录 | 模型 | 上下文剩余百分比

如需调整请告诉我。

* Cooked for 51s

* recap: 您刚完成了 statusLine 配置，显示当前目录+模型+上下文剩余百分比。如果需要调整或还有其他需求，请告诉我。 (disable recaps in /config)
```

步骤5:对话恢复

- 输入以下指令后，选择恢复到你想要的会话即可：

代码块

```
1 /resume
```

三、个性化

1. Claude.md 配置

项目级 Claude.md 创建方式：

代码块

```
1 /init
```

全局级 Claude.md 有两种创建方式：

方式 1：提示词交互

代码块

```
1 记得永远说中文，写进全局claude.md
```

方式 2：使用指令

输入 `/memory` 后，选择「User Memory」进入：

Memory

Auto-memory: on

- › 1. User memory Saved in ~/.claude/CLAUDE.md
- 2. Project memory Saved in ./CLAUDE.md
- 3. Open auto-memory folder

Learn more: <https://code.claude.com/docs/en/memory>

Enter to confirm · Esc to cancel

2. Auto Memory

- 打开Auto Memory：输入 `/memory` ，选择「Auto-memory」并输入回车开启

```
> /memory

Memory

> Auto-memory: on

1. User memory           Saved in ~/.claude/CLAUDE.md
2. Project memory        Saved in ./CLAUDE.md
3. Open auto-memory folder

Learn more: https://code.claude.com/docs/en/memory
```

打开以后，我们与CC的工作交互过程中，那些没有显式的主动写进claude.md的一些习惯、错误、经验，都会以自动记忆形式被记录，但仅限于当前项目，不会跨项目产生影响。

四、能力扩展

1. Skill 技能扩展

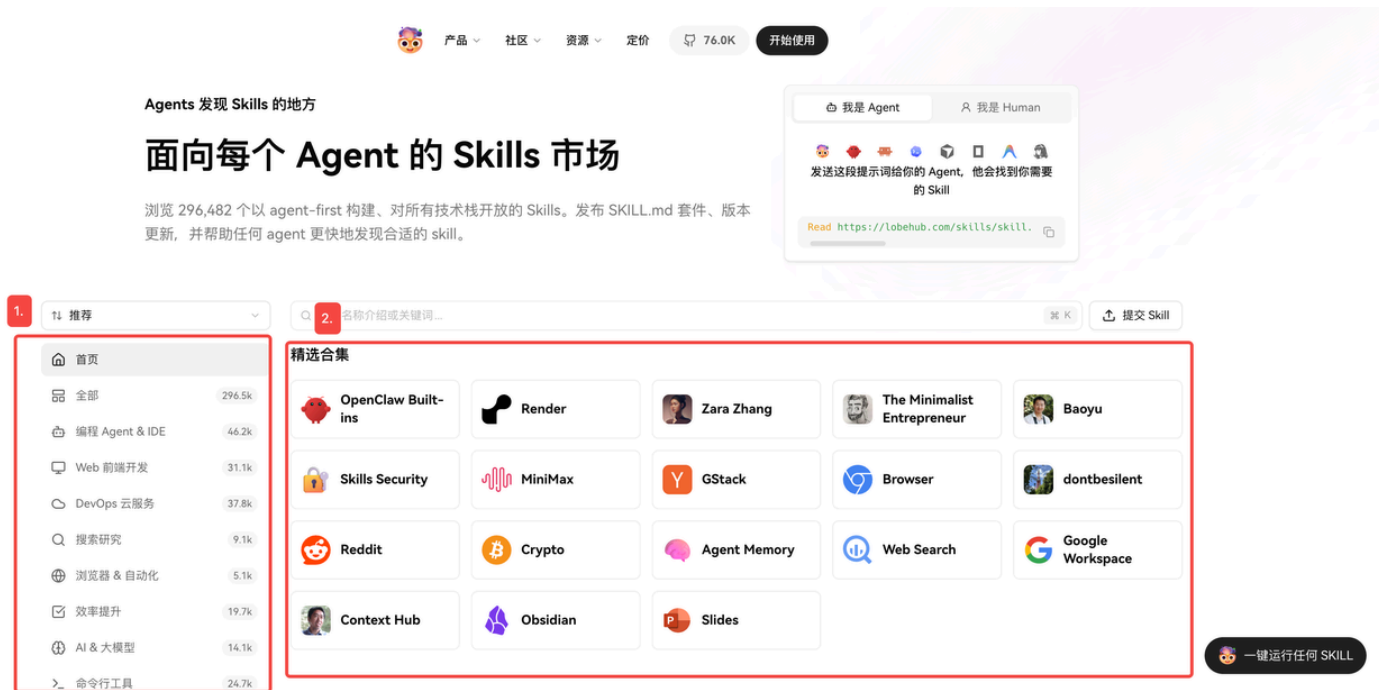
1. Claude Code优质skill推荐

- 可以将skill链接给cc，让cc帮你直接下好

skill名称	功能	下载链接
Find-Skill	根据用户需求，查找和安装来自agent skill开放生态的技能	GitHub
Frontend- Design	创建具有独特风格、生产级品质且设计精良的前端界面	GitHub
Skill- Creator	创建新skill、修改和改进现有skill，并衡量skill表现	GitHub
卡帕西skill	一个依据卡帕西经验总结，用于提升Claude Code编码表现的skill	GitHub

2. skill合集网站：lobehub

- 大家既可以根据分类去寻找自己需要的skill，也可以直接在精选合集查看推荐的优质skill



3. 想创建自己的skill?

- 我们还有一份网页版教学文档，非常详细：[Agent Skills指南（Claude Code版）](#)

4. 下载了skill怎么装?

- 可根据需求，将skill文件放入**项目级skill**与**全局级skill**的存放文件夹

● 全局和项目级 Skill 的位置

全局 Skills

全局 skills 放在你的 用户配置目录 下：

`/Users/qiu007/.claude/skills/`

这就是当前我所在的工作目录，也是你全局安装的 skill 存放位置。

项目级 Skills

项目级的 skill 通常放在当前项目根目录的 `.claude/` 文件夹中，比如：
`/path/to/your/project/.claude/skills/`

当你用 `/skill` 命令时，Claude Code 会：

1. 先查找项目级 skills (当前项目 `.claude/skills/`)
2. 再查找全局 skills (`~/.claude/skills/`)

项目级 skill 可以覆盖同名的全局 skill，这样不同项目可以有定制化的行为。

2. MCP 扩展

关于MCP的部分，可结合以下教程进行学习与实践：

学习资源

 视频教程：用神器Claude Code！打造贴身AI秘书团【小白教程】

 文档教程：Claude Code教程

3. CLI 命令行工具

推荐几个优质CLI，直接将下载链接给CC，让CC全权负责下载和引导操作：

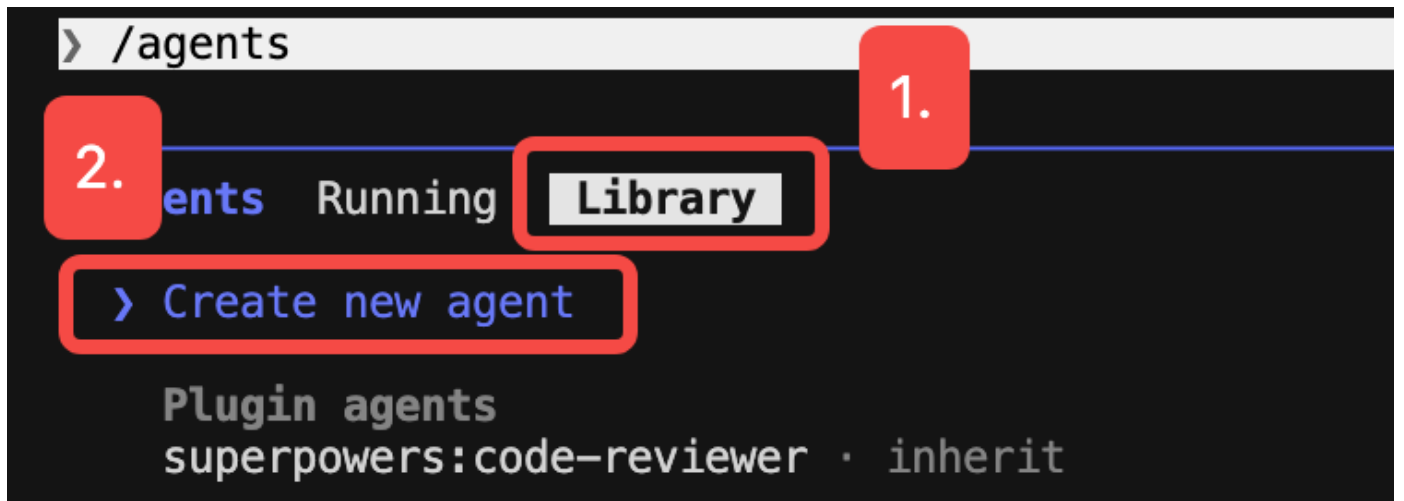
CLI名称	功能	下载
飞书CLI	飞书官方CLI工具，覆盖消息、文档、多维表格、电子表格、幻灯片、日历、邮箱、任务、会议等核心业务域，提供200+命令及24个AI Agent Skills	GitHub
OpenCLI	万能命令行工具箱，通用命令行中心与AI原生运行平台，能将任何网站、桌面应用或本地程序变成统一命令行操作界面	GitHub
CLI	GitHub 的官方命令行工具。它将拉取请求、问题和其他 GitHub 概念带到终端中，与你已经在使用 <code>git</code> 和代码的地方并排显示。	GitHub
gemini-CLI	Gemini CLI 可将 Gemini 的功能直接引入终端。它提供轻量级的 Gemini 访问方式，能够以最直接的方式从终端命令访问 Gemini 模型。	GitHub

给大家推荐一个GitHub上的CLI主题推荐网页，大家可按需查找自己想要的CLI：[Command-line interface](#)

4. 子Agent（SubAgent）

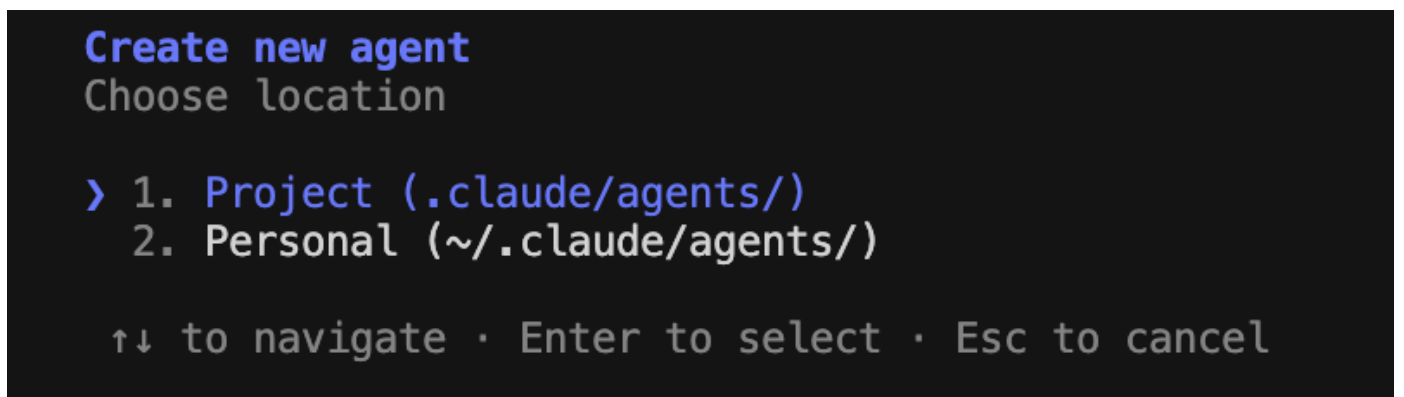
创建子Agent的两种方式：

- 1. 自动触发：任务复杂且存在并行可能时，CC会自动派生子Agent并行推进
- 2. 手动创建：通过指令 `/agents` ，在Library界面进行创建

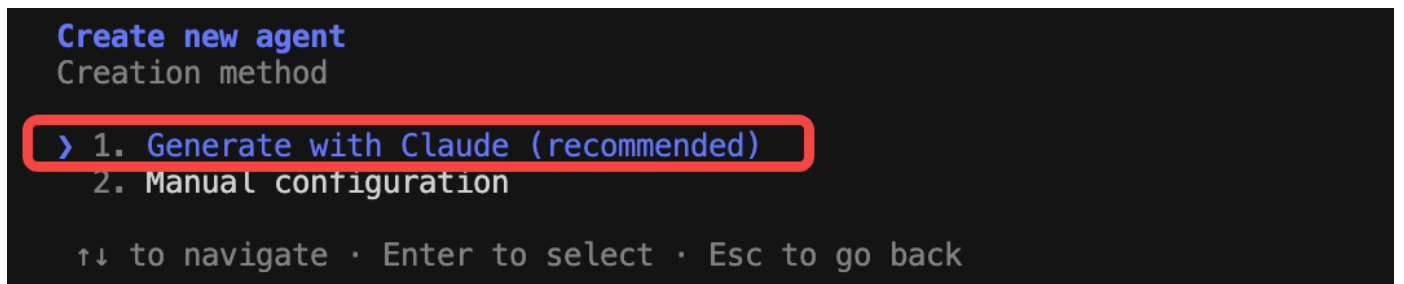


手动创建步骤：

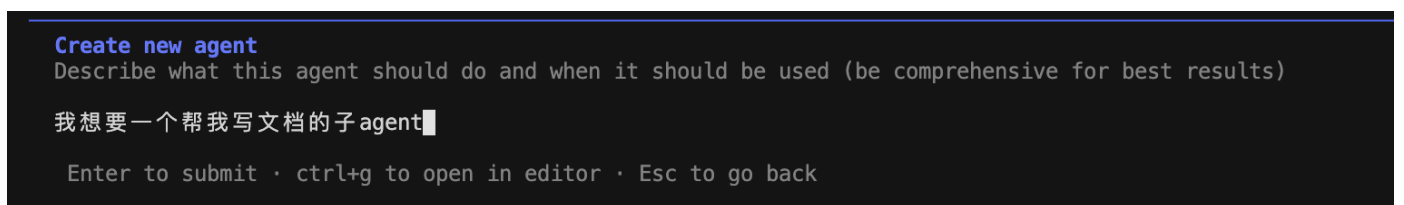
步骤1：选择创建项目级或全局级子Agent



步骤2：选择「AI辅助创建」，让AI根据意图辅助创建



步骤3：描述想要的子Agent功能



步骤4：决定子Agent工具权限（√为选中）

Create new agent

Select tools

> [Continue]

- ☒ All tools
- ☒ Read-only tools
- ☒ Edit tools
- ☒ Execution tools
- ☒ MCP tools
- ☒ Other tools

[Show advanced options]

All tools selected

Enter to toggle selection · ↑↓ to navigate · Esc to go back

步骤5：选择Claude的模型

- 一般选Sonnet就可以（如果使用CC Switch配置了模型，选哪个都一样）

Create new agent

Select model

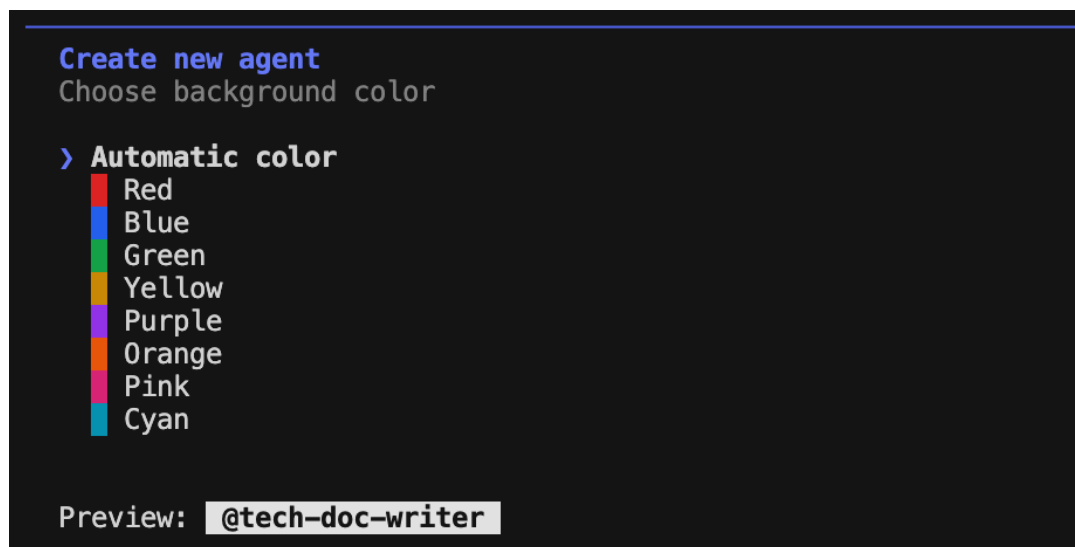
Model determines the agent's reasoning capabilities and speed.

- > 1. Sonnet ✓ Balanced performance – best for most agents
- 2. Opus Most capable for complex reasoning tasks
- 3. Haiku Fast and efficient for simple tasks
- 4. Inherit from parent Use the same model as the main conversation

↑↓ to navigate · Enter to select · Esc to go back

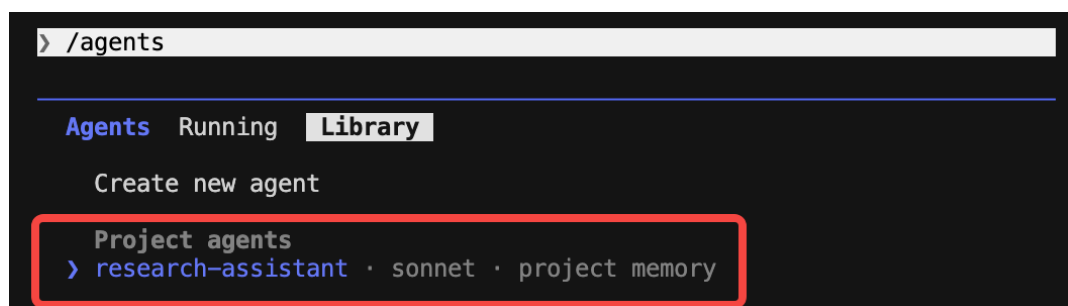
步骤6：为子Agent挑选区别于主Agent的颜色

- 选个好看的颜色

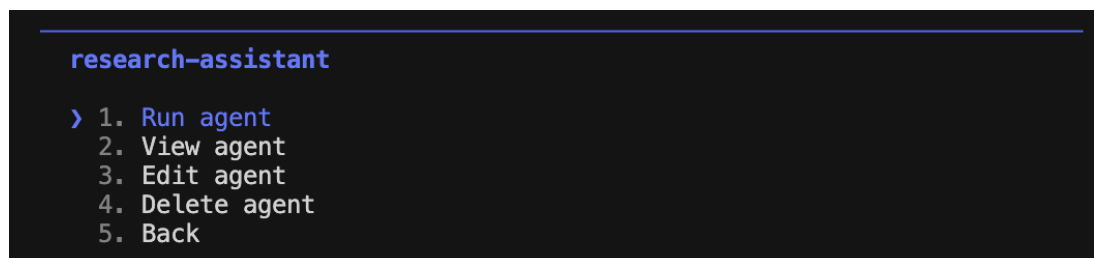


步骤7:调用与管理子Agent

- 输入 `/agents` ，在Library下选择已创建的项目级子Agent



- 根据需求对子Agent进行相应操作



5. Hook 钩子

推荐以下Hook设置，直接告诉CC帮忙设置：

推荐1：任务完成提示音

- 1 设置一个hook，每次完成任务之后，都自动执行一个声音脚本，发出一个提示音"叮"进行提醒

推荐2：代码提交前检查

- 1 设置一个hook，每次提交代码之前，都会自动触发代码格式的检查

6. 插件（Plugin）

插件说明

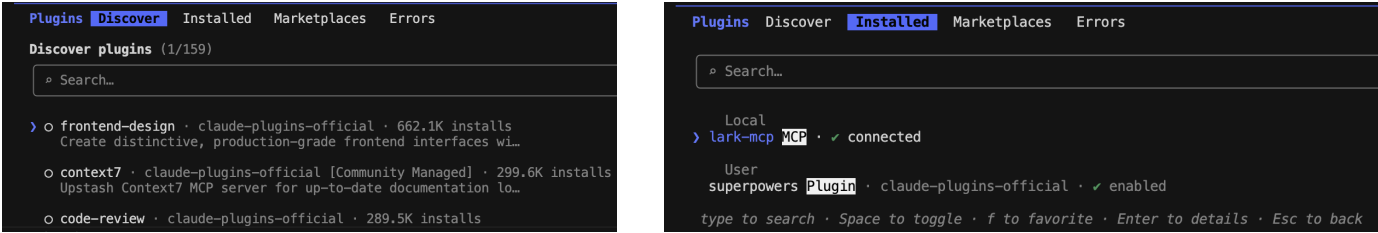
插件是打包了 Skill、SubAgent、Hook、MCP 的整合性概念

步骤1：通过指令 `/plugin` 进入插件管理界面

```
> /plugin

/plugin (plugins)      Manage Claude Code plugins
/reload-plugins        Activate pending plugin changes in the
                        current session
```

步骤2：在插件管理界面，可以收录下载钟意的插件，或管理已下载插件



推荐给大家一些Claude Code官方精选的一些插件：

插件名称	插件功能	链接
commit-commands	使用简单的命令简化 Git 工作流程，包括提交、推送和创建拉取请求	GitHub
content-creator	这位内容创作者擅长跨平台内容创作，从长篇博客文章到引人入胜的视频脚本和社交媒体内容，无所不包。	GitHub
security-guidance	安全提醒钩子，在编辑文件时提示潜在的安全问题，包含命令注入、XSS 及不安全的代码模式	GitHub

如果仍有需求，可以去官方/三方插件网站继续查看自己需要的插件：[Plugins / Claude Code Plugins](#)

(注：内容由 AI 生成，请谨慎参考)